APPLIED CRYPTOGRAPHY

RSA - ASSIGNMENT

Emmanuel Antwi Nyamaah

Contents

Introduction 2
Steps to Implement the RSA Code: 2
Figure 1- Libraries 2
Figure 2 – Key Generations 2
Figure 3 - Encryption 3
Figure 4 - Decryption 3
Figure 5 – Testing decryption 3
User Interface 4
Figure 6 – User Interface1 4
Figure 7 - User Interface2 4
Outputs 5
Figure 8 – Interface options outputs 5
Figure 9 – RSA key generation 5
Figure 10 -Encrypted Message 6
Figure 11 - Decryption 6
Figure 12 – Testing decryption 1 6
Figure 13 - Testing decryption 1 6
Figure 14 – End of program 6
Conclusion 7
References 8

Introduction

In this report, we implement the RSA encryption algorithm in Python. Key Generation, File Encryption, Decryption Example, and a security test for decryption at the real end using the wrong key. This implementation makes use of the Crypto library in Python and incorporates features like the ability to interact with a clear menu of cryptographic operations, Automatic key generation, and clear display of output.

Steps to Implement the RSA Code:

Firstly, I needed to import the necessary Libraries; Crypto.PublicKey.RSA, which was for generating RSA key pairs and Crypto.Cipher.PKCS1_OAEP which was used for encryption and decryption. This also used RSA with Optimal Asymmetric Encryption Padding(OAEP). OS, random, and string were for general random filenames for keys and for systems operations like reading and writing files. These all derived from me downloading the "pip install pycryptodome" on my command prompt.

Figure 1- Libraries

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
import os
```

Next, I generated a 2048 key size with the generate_rsa_keys() function. I then exported the keys using the key.publickey().export_key() and key.export_key(), respectively. After this, I stored the keys in separate files with unique filenames using random.choices() and string.ascii_letters + string.digits for enhanced security.

Figure 2 – Key Generations

```
def generate_rsa_keys(key_size=2048):
  """Generates RSA public and private keys."""
  key = RSA.generate(key_size)
  private_key = key.export_key()
  public_key = key.publickey().export_key()

  print(f"\nPublic Key:\n{public_key.decode()}")
  print(f"\nPrivate Key:\n{private_key.decode()}")

  with open("private_key.pem", "wb") as f:
    f.write(private_key)

  with open("public_key.pem", "wb") as f:
    f.write(public_key)
```

Thirdly, the encryption function, where we encrypt the message. Firstly, I loaded the public key, created cipher objects using the public key. encrypted the plaintext using the cipher.encrypt() method and wrote the encrypted ciphertext to a new file with the ".enc" extension

Figure 3 - Encryption

```
def encrypt_file(file_path, public_key_path):
  """Encrypts a file using the provided public key."""
  with open(public_key_path, "rb") as f:
    public_key = RSA.import_key(f.read())

  cipher = PKCS1_OAEP.new(public_key)

  with open(file_path, "rb") as f:
    plaintext = f.read()

  ciphertext = cipher.encrypt(plaintext)

  print(f"\nEncrypted Message (bytes):\n{ciphertext}")

  with open(file_path + ".enc", "wb") as f:
    f.write(ciphertext)
```

For decryption, much like encryption, load private keys instead using the function RSA.import_key(). Using the private key, create a PKCS1_OAEP cipher object and attempt to decrypt the ciphertext using cipher.decrypt().

Figure 4 - Decryption

```
def decrypt_file(file_path, private_key_path):
  """Decrypts an encrypted file using the provided private key."""
  with open(private_key_path, "rb") as f:
    private_key = RSA.import_key(f.read())

  cipher = PKCS1_OAEP.new(private_key)

  with open(file_path, "rb") as f:
    ciphertext = f.read()

  try:
    plaintext = cipher.decrypt(ciphertext)
    print(f"\nDecrypted Message:\n{plaintext.decode()}")
    print("Decryption Successful and completed.")
  except ValueError:
    print("Decryption Failed. Incorrect key or corrupted data.")
```

For the intrusion detection mechanism, read a different key file(not the correct private key) using RSA.import_key(). Expect and handle the expected ValueError that occurs when decryption fails with the wrong key and then the message suggesting the failure or success of the decryption is printed.

Figure 5 – Testing decryption

```
def test_decryption_with_incorrect_key(file_path, incorrect_key_path):
  """Attempts decryption with an incorrect key."""
  try:
    with open(incorrect_key_path, "rb") as f:
      incorrect_key = RSA.import_key(f.read())

    cipher = PKCS1_OAEP.new(incorrect_key)

    with open(file_path, "rb") as f:
      ciphertext = f.read()

    plaintext = cipher.decrypt(ciphertext)
    print("\nDecryption with Incorrect Key: Successful ;(")
  except ValueError:
    print("\nDecryption with Incorrect Key: Failed ;)")
```

User Interface

I created a simple user interface where all the required actions can be seen. This interface is designed to be simple and user-friendly, providing a clear and concise interaction for the user. At the beginning of every interaction cycle, the program presents the user with a menu of options. For user input handling, the program utilizes the input() function to keep user input for menu selection and file paths. The program also provides informative messages at various execution stages such as “RSA keys generated successfully” and “Encryption completed”.

Figure 6 – User Interface1

```
if __name__ == "__main__":
  while True:
    print("\nChoose an action:")
    print("1. Generate RSA Keys")
    print("2. Encrypt File")
    print("3. Decrypt File")
    print("4. Test Decryption with Incorrect Key")
    print("5. Close Program")

    choice = input("Enter your choice (1-5): ")
```

Figure 7 - User Interface2

```
if __name__ == "__main__":
  while True:
    print("\nChoose an action:")
    print("1. Generate RSA Keys")
    print("2. Encrypt File")
    print("3. Decrypt File")
    print("4. Test Decryption with Incorrect Key")
    print("5. Close program")

    choice = input("Enter your choice (1-5): ")

    if choice == '1':
      print("Generating RSA keys...")
      generate_rsa_keys()
      print("Keys generated successfully.")

    elif choice == '2':
      file_path = input("Enter the path to the file to encrypt: ")
      print("Encrypting file...")
      encrypt_file(file_path, "public_key.pem")
      print("Encryption complete.")

    elif choice == '3':
      file_path = input("Enter the path to the encrypted file: ")
      print("Decrypting file...")
      decrypt_file(file_path, "private_key.pem")
      print("Decryption complete.")

    elif choice == '4':
      print("Testing decryption with an incorrect key...")
      incorrect_key_path = input("Enter the path to the incorrect key path: ")
      test_decryption_with_incorrect_key("Ghana.txt.enc", incorrect_key_path)

    elif choice == '5':
      print("Program Exited. Goodbye")
      break

    else:
      print("Invalid choice. Please enter a number between 1 and 5.")
```

Outputs

When we run the code, this is the first thing we see. This is what the interface looks like. As we saw earlier in the code, we are presented with five options, and the user is able to enter a choice from one to five.

Figure 8 – Interface options outputs

```
Choose an action:
1. Generate RSA Keys
2. Encrypt File
3. Decrypt File
4. Test Decryption with Incorrect Key
5. Close program
Enter your choice (1-5):
```

When the user enters 1, the RSA keys are generated, and we are presented with a private and public key.

Figure 9 – RSA key generation

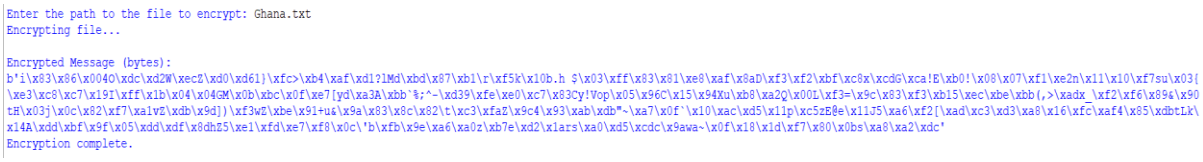
```
Generating RSA keys...

Public Key:
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAtRwVb5V8tURsMuXdEcHG
hiiybivTTqKvcsIfArkn8dYCGaQj76blD4D1xA++Nc/3SyjKw2MYJKgHRX5+VOpY
vohrHH72SgKwfFkQ/w95QecY3B2NmsN8dNkBDkHQ0gZOpSYq0nXEf9WAc42s6RZ9
I/i17i/+BboIXCtl+DOfx4yahbHoIviQsg9gZbulKWH7HT6ZDkZO6rDG/7LWTkJQ
geRIt6pYZddk0NV6thZ+z+ZoaoYATOO0XOuwVqv8B1HNQX4Nh/H5GyncvelRci4O
Y9XeLhgjCWnzLl2kEdApAoNiviyCmGQSQRq1YwcA9Ijfh8qPz26ghO+RkTVIRKDk
jwIDAQAB
-----END PUBLIC KEY-----

Private Key:
-----BEGIN RSA PRIVATE KEY-----
MIIEogIBAAKCAQEAtRwVb5V8tURsMuXdEcHGhiiybivTTqKvcsIfArkn8dYCGaQj
76blD4D1xA++Nc/3SyjKw2MYJKgHRX5+VOpYvohrHH72SgKwfFkQ/w95QecY3B2N
msN8dNkBDkHQ0gZOpSYq0nXEf9WAc42s6RZ9I/i17i/+BboIXCtl+DOfx4yahbHo
IviQsg9gZbulKWH7HT6ZDkZO6rDG/7LWTkJQgeRIt6pYZddk0NV6thZ+z+ZoaoYA
TOO0XOuwVqv8B1HNQX4Nh/H5GyncvelRci4OY9XeLhgjCWnzLl2kEdApAoNiviyC
mGQSQRq1YwcA9Ijfh8qPz26ghO+RkTVIRKDkjwIDAQABAoIBAASCFckt8BQVse51
E6hU1HuTw8KY9gI9FnTwKWlzRm/vTc9pleNp73BlW2NtU49vWnkqeMby+S+LEd3u
+iIO+0qIMXJczrCk7ykJO90S0N03wsH9PQB0i+00rEwN41l3ArolqxHgu0AbW76E
m6CUXg++fyOAr6gLgMMalB02MRdRzo7HlYBxkrCsPb0+LnX2qhdkVHso/6sI/SfR
wuuIm78YbVeCC8ZdxBm/YD8p6e1aP9JBdGYLEbVAE5swECyWkAKrEZOcg+y3V6ZE
aQq86NVh5ClYxh3hA8Kj0/0xtXoTqwQMKFq+YAdOgc+f/z2qGPLEKWBREE9yZOPw
qn4sSDECgYEAwzuW7nuKdywX7TbcVJZAy3RKdmBnwdziiXfVLnxqCCdW6Sm14V3O
PfPRVwJE+iGFhWs8jfcdnB1rV6npd46ra8lt+aMQVXOIjPfTYBAJs/xZvL65XTE5
HljZKQuk8zago458ay6gkMtXPMKAeqm8R78Z7O2yf7zOKSdo4StLnC0CgYEA7Xsm
IAtheOVw84aN9/DLmaEKiyr4KDJsZIfuYguq/c7BZ4QB9/XnfwcnmxdjtSE4okOc
g+KRgs21/E7dZlf/hp/bhWcC+wbXwpnSrWkotbCaipdKxTgNbCptXazfXInmwpQl
aN+of2rnY5QAX4g1iHqTUiV1TtaP1Nl+0Bu47SsCgYAos6CowWrBo5itZLWUkOqb
P/hSK5EWEpXL8QaptVhpsMHmxEJd5K+drNok2J1GxHxQr2WldzJeHKgHU64Ptde5
Ffpug/dgyUBbm0MfENjlBCQ1PFHqwttUn1NOwlW2OtpMnuUDd2oUJcBfM3GToRVW
o9mk0n1XMiaM3+F2qA8QVQKBgBNAlZGpeohziVmkweOk2MymmpU3sZCIrfLkdjXq
nj0fQtyifgpdHBxXNyXCmqm3fMDi3FBw0L0a8jL3Lcm7PpLqaMCC3mH9ZwZ2s/as
/Mjl/VFY/I3KgoQPWHhsqFr3Lwb9Pz8n6qARyZpwCCu2fiiHT2Qsdzsh990YmMhE
RxSrAoGAD2x0csUVpDFhLgg3uLJ0eI4WQ5lnNQbrojkNy81cGGEzThVbqaDGmI85
Wgnj/kjMfb0v6T7vTWwAHYhqDEA5YRATZwbq81etY/tp9BsJ+LR0hLkyjydcoMoW
eeelyqFIx1kAgDyeDi7QJC3s7iB+/wzHjIi5VwQiwEE2EyDpTE8=
-----END RSA PRIVATE KEY-----
Keys generated successfully.
```

When 2 is entered, we get a prompt to enter the file we'd like to encrypt. When we enter the file, the file is then encrypted, and we get a ciphertext.

Figure 10 -Encrypted Message



To decrypt the message, we are prompted to enter the path of the encrypted file which then decrypts the message.

Figure 11 - Decryption

```
Enter the path to the encrypted file: Ghana.txt.enc
Decrypting file...

Decrypted Message:
I love Ghana
Decryption Successful and completed.
Decryption complete.
```

When we enter the 4th option, it tells us to enter an incorrect key path to test the decryption and this is the result of entering the incorrect key path.

Figure 12 – Testing decryption 1

```
Enter the path to the incorrect key path: wrongkey.pem

Decryption with Incorrect Key: Failed ;)
```

And here is the write key path entered which decrypts the message but that's not what we want.

Figure 13 - Testing decryption 1

```
Enter the path to the incorrect key path: private_key.pem

Decryption with Incorrect Key: Successful ;(
```

The program exists when we enter 5.

Figure 14 – End of program

```
Enter your choice (1-5): 5
Program Exited. Goodbye
>>>
```

Conclusion

This report describes a successful implementation of the RSA encryption algorithm using a Python program. It includes code for key generation, file encryption, file decryption, and a security test that validates that decryption fails when an incorrect key is used.

The security test, which eschews successful decryption attempts with the wrong key, effectively illustrates the desired failure, reassuring the integrity of the RSA encrypting scheme. Overall, this project aimed to build an understanding of how RSA encryption worked conceptually and practically.

Overall, this project is a great learning experience in cryptography and allows for further experimentation to implement more sophisticated cryptographic primitives.

References

Dirog (2021). *Python library for RSA public decryption*. [online] Stack Overflow. Available at: https://stackoverflow.com/questions/68458019/python-library-for-rsa-public-decryption.

Berry, A. (2021). *Generating Encrypted Key Pairs In Python*. [online] DEV Community. Available at: https://dev.to/aaronktberry/generating-encrypted-key-pairs-in-python-69b [Accessed 19 Dec. 2024].

David (2014). *How do I encrypt and decrypt a string in python?* [online] Stack Overflow. Available at: https://stackoverflow.com/questions/27335726/how-do-i-encrypt-and-decrypt-a-string-in-python [Accessed 24 Dec. 2024].

Abdeladim Fadheli (2019). *How to Encrypt and Decrypt Files in Python - The Python Code*. [online] Thepythoncode.com. Available at: https://thepythoncode.com/article/encrypt-decrypt-files-symmetric-python [Accessed 6 Jan. 2025].

Lex, D. (2022). *RSA Encryption & Decryption In Python: Key Creation, Storage, Algorithm, and Forward Secrecy*. [online] onboardbase.com. Available at: https://onboardbase.com/blog/rsa-encryption-decryption/ [Accessed 6 Jan. 2025].